Parmana Technical Guide

How Parmana works, how policies and human approvals control what an AI agent may do, how to verify the signed records it leaves, and what it does not guarantee.

Source: the `docs/site/build-book` chapters of the `parmana-core` repository. Where this guide and the code disagree, the code is right.

Scope

This guide covers the request lifecycle, core concepts, policies, human approvals, verification and audit, and the security model with its stated limits. It does not cover production deployment or troubleshooting. For those, see the full "Build with Parmana" book and the online documentation at docs.parmanasystems.com.

1. What Parmana does

Parmana sits between an AI agent (or an application or a person) and the systems the agent acts on. Three questions stand between an agent and production:

- **What may this agent do?** A rule written in a prompt is not a control. The agent, or text injected into it, can ignore it.
- **Who approved it?** A decision made inside the agent's own process is an assertion, not a control.
- **Can we prove what happened?** A log written by the system that acted is a claim, not evidence.

Parmana answers each one:

- Every action is checked against an explicit, versioned policy that two people approved.
- Actions that change state need a signed approval from a trusted person, for that action and that resource.
- The agent never receives the credentials of the system it acts on. An approved action is released once, through a connector.
- Every decision, approved or refused, leaves a signed record that anyone can verify offline.

Parmana does not decide what the agent should do. It decides whether the agent was allowed to do it, and it records that decision.

2. How one request moves through Parmana

Step	What happens
1	The agent asks which policy governs the action: `GET /policies/in-effect?capability=...`. The answer names the policy, its version and the signals a request must carry.
2	The agent sends a **Business Transaction** to `POST /execute`: who is acting (authority), for what purpose (authorization), what to do (intent: action, target, parameters), the policy it declares, and its signals.
3	Parmana checks that the caller's key may use the action, and that the declared policy is the one bound to the action at its version in effect.
4	Signals bound to the request are checked against it. An approval signal counts as true only with a valid signed approval from a trusted person for that action and resource.
5	The policy's rules run in order. The first rule that matches decides. The same inputs always give the same decision.
6	**Refused:** Parmana returns the reason and stores a signed **Refusal Record**. Nothing is released.
7	**Approved:** Parmana signs an **execution authorization**. It is single use, time bounded, and bound to the exact content approved.
8	Before release, Parmana stores a signed **Execution Intent**: exactly what is about to be released.
9	The **gateway** checks the authorization again (signature, expiry, content, single use, policy, signals and approval) and releases the action to the connector for that action.
10	The connector acts, using a credential only it can use, for this one execution.
11	Parmana signs an **Execution Trust Record**: the transaction, the decision, the authorization and the connector's evidence.
12	Anyone with the public key can verify the record offline, without trusting the server.

Principles

- **No action without a person.** Every policy must require a signed human approval for any action that could be approved, reads included. The server refuses to load a policy that could approve without one. Plan for the first request on a new resource to be refused until a person signs.
- **The agent cannot authorize itself.** Facts the agent declares can refuse an action. They can never authorize one on their own. Only a signed approval, or a fact the server checks itself, can make an approve rule match.
- **Parmana releases the action; the agent does not.** An agent that performs the action itself after asking Parmana defeats the control. The agent only reads the answer.
- **Changes need two people.** A new policy version, a new approver key and a new external connector each need one person to propose and a different person to approve. No deploy is needed.

- **Everything is evidence.** Approvals, refusals, intents and results are signed and kept. A refusal is as provable as an approval.

3. Core terms

Term	Meaning
Capability, action	What the agent wants done, written `namespace:verb`, for example `erp:invoice-pay`.
Business Transaction	One request to act, with its identifiers, intent, policy and signals.
Signal	A fact in the request that a policy reads, such as `refundAmount` or `managerApproved`.
Approval signal	A signal that counts as true only with a signed approval.
Policy in effect	The version of the policy bound to an action, the most recently approved one.
Maker, checker	The person who proposes a change, and the different person who approves it.
Connector	The part that performs an approved action on a system: a built-in connector, or an external endpoint you register.

4. Policies

A policy is a JSON file that decides one kind of action. Its rules read the request's signals. The first matching rule decides approve or reject. A policy authorizes nothing until one person proposes it and a different person approves it.

4.1 Shape

The real `customer-refund` 1.2.0 policy has these parts, shortened here:

```
{
  "policyId": "customer-refund",
  "policyVersion": "1.2.0",
  "schemaVersion": "1.0.0",
  "signalsSchema": {
    "refundEligible": "boolean",
    "managerApproved": "boolean",
    "fraudCheckPassed": "boolean",
    "refundAmount": "number"
  },
  "boundSignals": { "refundAmount": "parameters.amount" },
  "approvalSignals": {
    "managerApproved": { "resourceId": "parameters.orderId", "value": "parameters.amount" }
  },
  "unboundSignalReasons": {
    "refundEligible": "Declared by the caller; it can refuse a refund, never authorize one on
its own."
  },
  "rules": [
    {
      "id": "reject-fraud-check",
      "condition": { "fact": "fraudCheckPassed", "operator": "is_false" },
      "outcome": { "action": "reject", "reason": "Refund rejected because the transaction did
not pass fraud assessment." }
    },
    {
      "id": "approve-refund-with-manager-approval",
      "condition": {
        "all": [
          { "fact": "managerApproved", "operator": "is_true" },
          { "fact": "refundEligible", "operator": "is_true" },
          { "fact": "fraudCheckPassed", "operator": "is_true" },
          { "fact": "refundAmount", "operator": "gt", "value": 0 },
          { "fact": "refundAmount", "operator": "lte", "value": 100000 }
        ]
      },
      "outcome": { "action": "approve", "reason": "Refund authorized with a verified manager
approval for this order and amount." }
    },
    {
      "id": "reject-manager-approval-required",
      "condition": { "always": true },
      "outcome": { "action": "reject", "reason": "Refund rejected. Every refund needs a signed
manager approval." }
    }
  ]
}
```

A policy lives at `policies/<policyId>/<policyVersion>/policy.json`.

Field	Required	Purpose
`policyId`	yes	The policy's name. Agents send it as `policy.name`.
`policyVersion`	yes	Its version. A change is a new version in a new folder, never an edit of an approved one.

Field	Required	Purpose
`schemaVersion`	yes	The policy format, `1.0.0`.
`description`	no	What it allows, in words. Returned by `GET /policies/in-effect`.
`signalsSchema`	no	The type of each signal: `boolean`, `number` or `string`. A request whose signal has another type is refused before any rule runs.
`boundSignals`	no	Signals that must equal a field of the request.
`approvalSignals`	in practice yes	Signals that count as true only with a signed approval. An approve rule needs one.
`unboundSignalReasons`	no	Why a signal the rules read is neither bound nor an approval signal.
`rules`	yes	At least one rule, each with a unique `id`, a `condition` and an `outcome`.

4.2 Rules and conditions

Rules run in order and the first match decides. If no rule matches, the request is refused (`no_rule_matched`). End every policy with an `always` rule that rejects with a reason the agent can act on.

An outcome is `{ "action": "approve" | "reject", "reason": "..." }`. The reason is returned to the agent and stored in the record, so write it for the person who reads it.

A condition is one of:

Condition	Matches when
`{ "fact": "x", "operator": "...", "value": ... }`	The operator holds for signal `x`
`{ "all": [...] }`	Every child matches
`{ "any": [...] }`	At least one child matches
`{ "always": true }`	Always

Operators, by group:

Group	Operators
Equality	`eq`, `neq`
Numbers	`gt`, `gte`, `lt`, `lte`, `between`
Lists	`in`, `not_in`, `contains`, `not_contains`, `contains_all`, `contains_any`
Text	`starts_with`, `ends_with`, `matches` (regular expression)
Presence	`exists`, `not_exists`, `is_null`, `is_not_null`

Group	Operators
Booleans	`is_true`, `is_false` (only the values `true` and `false`, never "truthy")
Length	`length_eq`, `length_gt`, `length_gte`, `length_lt`, `length_lte`

4.3 Every signal must be accountable

A signal is a claim. The policy must say how each signal its rules read is held to account, or the policy does not load:

The signal is	Declare it in	What the server does
A copy of a request field	`boundSignals`	Refuses the request unless the signal equals that field.
A person's decision	`approvalSignals`	Treats it as true only with a valid signed approval for this action, resource and amount.
Anything else	`unboundSignalReasons`	Nothing. The reason is recorded so a reviewer sees the gap. Such a signal should only ever refuse.

4.4 The rule the validator enforces

Every rule whose outcome is `approve` must require an approval signal with `is_true`, either as its whole condition or directly inside its top-level `all`. A nested `any` does not count, because another branch could match. A policy that breaks this does not load, and every request under it is refused with:

```
Policy rule '<id>' approves without a signed human approval. Every approve rule must require an
approvalSignals fact
with is_true, as its whole condition or directly inside its top level 'all'.
```

The design consequence: **the agent's own facts can refuse, but only a person can approve.** Put facts the agent declares in `unboundSignalReasons`, and use them in reject rules and as extra conditions next to the approval.

4.5 Check a policy before proposing it

Run the reference policy test, which loads every file under `policies/` through the same validator the server uses:

```
npx vitest run packages/policy/tests/unit/ReferencePolicies.test.ts
```

Message	Fix
`Policy references fact(s) 'x' with no boundSignals entry and no unboundSignalReasons entry`	Bind it, make it an approval signal, or give a reason.

Message	Fix
`approves without a signed human approval`	Add `{ "fact": "<approval signal>", "operator": "is_true" }` to the approve rule's top-level `all`.
`unboundSignalReasons['x'] is contradictory`	A bound signal needs no reason. Remove one of the two entries.
`Duplicate policy rule id 'x'`	Rule ids are unique within a policy.

4.6 From a file to the policy in effect

A policy takes effect when a second person approves it. No deploy is needed.

Step	Who	What
1	Author	Writes `policies/<name>/<version>/policy.json` and checks it.
2	Maker	`POST /policies/<name>/<version>/pending-changes` with `reason` and `proposedContent`. Answer: `201`, a `pendingPolicyChangeId`, status `PENDING_APPROVAL`.
3	Checker	Lists pending changes with `GET /policies/pending-changes?status=PENDING_APPROVAL` and reads the content.
4	Checker	Signs a step-up authorization on their own machine and posts it to `.../<id>/approve` within 120 seconds.
5	Server	Checks the checker is a human, is not the maker, and that the step-up signature is valid. Writes a signed, chained approval record and saves the policy.
6	Everyone	From the next request on, the new version is the one in effect.

Maker and checker each need an API key registered as a human (`credentialHolderType USER`). The checker also needs a registered step-up public key. The SDKs provide the same flow: `proposePolicyChange`, `approvePolicyChange` and `rejectPolicyChange` in TypeScript, and `propose_policy_change`, `approve_policy_change` and `reject_policy_change` in Python.

4.7 Change and roll back

- **Change.** Copy the folder to a new version, edit, check, propose and approve. Agents that read the policy in effect follow at once. A request naming an old version is refused before any rule runs, with a message naming the version in effect.
- **Roll back.** Propose and approve the older version again. The most recent approval decides the version in effect.
- **Keep the file unchanged.** The approval record stores a hash of the approved content. CI's `verify-policy-approvals` job and the server at startup compare that hash with the file in Git. One changed character fails the check. `npx tsx scripts/verify-policy-changes-approved.ts --full-scan` runs the same comparison.

A new action for your own system needs no deploy: register it as an external connector, with its policy, through maker-checker. Binding a built-in action to a different policy name does need a deploy.

4.8 What is checked on every request

- The policy named is the one bound to the action, at the version in effect.
- The policy has an approval record, the record's signature verifies, and its content hash equals the live content. A policy edited in the database outside this flow authorizes nothing.
- The gateway checks the approval record again just before release.

These checks are always on in production. In `NODE_ENV` `test` or `development` they are off unless `POLICY_EXECUTION_VERIFICATION_ENFORCED=true` is set, which is why the local example server accepts unapproved example policies.

5. Human approvals

No agent action runs without a person. A policy names an **approval signal**. The agent may set it to `true` only with a **signed approval** from a trusted approver, for this action and this resource (and amount, where the policy names one), not expired, and used once.

5.1 Three roles, three kinds of key

Role	Signs	Key	Registered through
Action approver	One action on one resource: "this refund, once"	Ed25519 approver key, held on their own machine	Maker-checker
Maker	Nothing; proposes changes	API key registered as a human	The operator
Checker	Approval of a change: a policy, an approver or a connector	API key registered as a human, plus a step-up key	The operator

Private keys never leave their owner's machine. Only public keys are sent to anyone.

5.2 Trust an approver

An approver is trusted when their public key is in the server's approver list. Adding one needs no code change and no deploy:

1. The approver generates a key pair on their own machine:

   ```
   npx tsx scripts/generate-approver-key.ts --approver-id approver-example --key-id approver-example-key-1 --out-dir ~/.parmana
   ```

   They keep the private `.pem` file and send only the `.public.pem` file.

2. A maker proposes the addition with `POST /approval-issuers/changes`, `action: "add"`, the ids, the public key and a reason.

3. A checker compares the public key with the approver over a separate channel, signs a step-up authorization for the change, and approves it with `POST /approval-issuers/changes/<changeId>/approve`.

4. From the next request on, approvals signed with that key verify. `GET /approval-issuers` lists every trusted key and where it came from.

To rotate, add a new key id, switch to it, then revoke the old one. Revoking refuses every approval the key ever signed, whether used or not. Key ids are never reused.

5.3 Sign an approval

The approver signs on their own machine, for one action, one resource and, where the policy names one, a maximum amount:

```
npx tsx scripts/sign-approval.ts \
  --private-key-file ~/.parmana/approver-example__approver-example-key-1.private.pem \
  --approver-id approver-example --key-id approver-example-key-1 \
  --capability erp:invoice-pay --resource-id INV-1042 --max-amount 75000 --out approval.json
```

The TypeScript SDK provides `signApproval()` and the Python SDK provides `sign_approval()`. An approval lasts 15 minutes by default. The signing tools refuse lifetimes over one day. The server checks only that the approval has not expired.

5.4 What the server checks

The agent sends a new request with the approval signal set to `true` and the approval in `signals.approvalArtifact`. Before deciding, and again at the gateway just before release, the server checks:

Check	If it fails
The approver and key are trusted and not revoked	The approval signal counts as `false`; the request is refused
The Ed25519 signature over the payload verifies	Same
`expiresAt` is in the future	Same
`capability` equals the request's action	Same
`resourceId` equals the request field the policy names	Same
The amount in the request (never the agent's own signal) is within `scope`	Same
The approval has not been used before (its nonce is stored once used)	Same

A refused approval is answered with `403 POLICY_DENIED`, because the declared signal does not match what the server verified.

5.5 How a person learns an approval is needed

A request refused for lack of an approval is a normal answer. Three ways to reach the approver:

- **Refusal Records.** Every refusal is stored and signed, with its reason and the request. Query them by policy and matched rule.
- **A webhook.** With `APPROVAL_WEBHOOK_URL` and `APPROVAL_WEBHOOK_SECRET` set, Parmana posts an `approval.needed` event, signed with HMAC SHA-256, only when the same request with the approval signals set to `true` would have been approved. Its `approvals` list is what to

sign.
- **Email.** With Resend configured and `APPROVAL_EMAIL_TO` set, the same event is emailed, at most once per refusal.

Webhook and email delivery are best effort: one attempt, no retry, and a failure never changes the refusal. The Refusal Records remain the complete list.

5.6 What an approval does not fix

- **A refused request is never approved later.** Nothing waits. After the approver signs, the agent sends a new request.
- **An approval covers the action and the resource, and the amount where the policy names one.** It does not fix other parameters. An approval for a message channel does not fix the message text.
- **Facts the agent declares remain its own word.** A policy should use them only to refuse.
- **There is no automatic path.** No policy can approve without a person, and the validator refuses to load one that tries.

5.7 Approving changes: the step-up authorization

Approving a policy, an approver or an external connector uses a different signature: a **step-up authorization** from the checker. It covers one change and one decision (`approve` or `reject`), is valid for at most 120 seconds, and can be used once. The server refuses it when the checker is the maker (`403 SAME_ACTOR_CANNOT_APPROVE_OWN_CHANGE`) or is not registered as a human (`403 NON_HUMAN_CALLER_DENIED`).

```
npx tsx scripts/sign-policy-change-step-up.ts --private-key-file step-up.private.pem \
  --key-id <checker caller id> --pending-policy-change-id <change id> --action approve
```

`scripts/local-review-action.ts` signs and submits in one process, so the 120 seconds cannot run out between the two steps.

5.8 Keep it honest

Parmana proves that a key signed. It cannot prove whose hand held the key. Two keys held by one person make every record claim a second person who does not exist. Keep each key on its owner's machine, and never paste a key into a chat or a ticket.

6. Verify and audit

Every decision leaves signed evidence. An auditor, a customer or your own agent can check it with Parmana's public key alone, with no account and no running server.

6.1 The records

Record	Written when	Proves	Read it with
Execution Trust Record	An approved action was released and answered	The transaction, the decision, the authorization and the connector's evidence	`GET /trust-records/{businessTransactionId}`
Refusal Record	A policy refused a request	The request and the refusal, with its reason and the policy content hash	`GET /refusal/{businessTransactionId}`
Execution Intent	Just before an approved action is released	Exactly what was about to be released	`GET /execution-intents/{businessTransactionId}`
Policy approval record	A checker approved a policy version	Who proposed, who approved, and the content hash before and after, chained to the previous record	`policy_change_approval_records` table
Caller audit events	A request was refused before any policy ran	Who called, for what, and why it was refused	`caller_audit_events` table

A Trust Record says "this ran, under this decision". A Refusal Record says "this was asked and refused". An Execution Intent says "this was about to run", which matters when the outcome is unknown.

6.2 Check a Trust Record offline

Fetch Parmana's public key once (`GET /keys/default`), keep it, and check any record against it. The TypeScript SDK provides `verifyExecutionTrustRecordOffline()` and the Python SDK provides `verify_execution_trust_record_offline()`. The result reports `valid`, `hashValid`, `legacySignatureValid` and the algorithms checked.

`valid` is true only when the record's hash equals a fresh SHA-256 of its canonical form **and** the Ed25519 signature verifies. Change any field, however deep, and both fail. `verifyExecutionIntentOffline` and `verify_execution_intent_offline` do the same for an Execution Intent.

When you build your own verifier, note three things:

- **Canonical form.** Keys are sorted and the bytes must be exactly what Parmana signed. Re-serializing the JSON through a tool that reorders keys or reformats timestamps breaks an untampered record. Keep the JSON exactly as received.
- **Large records.** A record over 4096 bytes of canonical content may be signed by AWS KMS over a fixed 97-byte commitment: the bytes of `PARMANA-ED25519-LARGE-MESSAGE-V1`, one NUL byte, then the SHA-512 digest of the canonical bytes. Try the raw signature first, and the commitment only for records over 4096 bytes. The SDK functions do this.
- **Hybrid signatures.** A record may also carry ML-DSA-65 signatures. The TypeScript verifier checks them; the Python verifier checks Ed25519 only.

The CLI script `scripts/verify-trust-record.ts` performs the same check from a terminal.

6.3 Check with the server

`verify(businessTransactionId)` asks the server to verify a record again and appends the result to the record's history. These answers trust the server. The offline functions do not, so prefer them when the question is whether to trust the server.

6.4 What a record proves, and what it does not

A valid Trust Record proves	It does not prove
Parmana's key signed exactly this content	That the key was not misused by whoever held it
The request, the decision and the policy version, as Parmana saw them	That facts the agent declared were true
The connector's evidence, as the connector reported it	That the system actually did what it reports. An external endpoint's answer is its claim.
The approval's issuer, as a key that verified	Whose hand held the approver's key

6.5 When the outcome is unknown

If a release fails after it may have reached the system (a timeout, a crash, or an error from the connector), the agent receives `502 EXECUTION_OUTCOME_UNKNOWN` and the Execution Intent stays open. **The action may have run.** Never send it again as a new transaction.

An operator closes the intent in three steps:

1. **List** open intents with `GET /execution-intents/unfinalized`.
2. **Decide by state:**

State	Meaning	Action
`RELEASED`	The connector answered, but the Trust Record was not stored	`POST /execution-intents/{id}/finalize`. It rebuilds the record without calling the connector, and is idempotent.
`PREPARED`	Stored, and the release may or may not have happened	Check the system, then resolve

State	Meaning	Action
`ERRORED`	The release raised an error; the action may still have run	Check the system, then resolve

3. **Resolve** after checking the system with the intent's `businessTransactionId`, `action` and `target`: `POST /execution-intents/{id}/resolve` with `resolution` set to `NOT_EXECUTED` or `EXECUTED`, and a required `note` saying what you checked. The `resolvedBy` field comes from your key.

A resolution is a person's attributed statement in the intent's unsigned status. It is an audit trail, not tamper-evident evidence.

6.6 Audit queries

Refused refunds, with their reasons:

```
select refusal_record_id, business_transaction_id, submitted_by, created_at,
       decision_json ->> 'reason' as reason
from refusal_records
where decision_json -> 'policy' ->> 'name' = 'customer-refund'
order by created_at desc
limit 50;
```

Requests refused before any policy ran:

```
select occurred_at, caller_id, capability, type, reason
from caller_audit_events
where type in ('caller.rejected', 'caller.capability_denied', 'caller.principal_denied',
               'caller.non_human_denied', 'caller.structural_rejected')
order by occurred_at desc
limit 50;
```

A Refusal Record is written after the refusal, and a failed write does not change the refusal. Each failed write is logged as `refusal_record_write_failed`. A query can therefore miss a refused request, and the logs are the backstop.

To compare every policy on disk with its latest signed approval record, run `npx tsx scripts/verify-policy-changes-approved.ts --full-scan`.

7. Security model and limits

7.1 What Parmana guarantees

For a system whose actions go through Parmana:

Guarantee	Enforced by
Only a key allowed the action can request it, and only for the principals it may act for	Caller authentication and the capability check, before any policy runs
Every action is decided by the policy bound to it, at the version two people approved	The capability policy binding and the policy approval record, checked on every request and again at the gateway
No action is approved without a signed approval from a trusted person, for that action and resource	The policy validator and the approval verifier, before deciding and at release
An approval is used once, within its expiry, for at most its amount	The approval verifier and its nonce store
What is released is exactly what was approved, once, within its lifetime	The signed execution authorization and the gateway's checks: signature, expiry, content hash, single use
The agent never holds a system's credentials	Connectors with per-execution credentials; external endpoints hold their own
Every decision, approved or refused, leaves a signed record anyone can verify offline	Trust Records, Refusal Records, Execution Intents, and chained audit events
Changes to trust need two human keys	Maker-checker with step-up signatures for policies, approvers and external connectors
Failures refuse	Every check fails closed. There is no weaker fallback.

7.2 What it assumes

- **Actions go through Parmana.** The gateway protects systems that route through it. An agent that also holds a system's credentials can act around it, so give agents no credentials.
- **Keys stay with their owners.** Parmana proves which key signed, never whose hand held it.
- **The signing key is safe.** A stolen signing key can sign records that verify. AWS KMS prevents the key from being released. Rotation is a manual procedure.
- **Policies are written well.** An over-broad rule approves what it should not, and no signature can detect that. The validator catches some mistakes, and two people review every version.
- **The operator is trusted with the database.** An operator with direct database access can change rows outside maker-checker. Such changes are recorded in those rows.

7.3 What Parmana does not claim

- That execution cannot be bypassed under any circumstances.

- A mathematical proof of correct execution.
- Regulatory compliance (SOC 2, GDPR, ISO 27001) by itself.
- That an external system did what its endpoint says. The answer is recorded as its claim.

7.4 Scoped limits to plan around

- **Single use is per nonce store.** Instances that share a database share the nonce store. Separate in-memory stores do not.
- **Route access is not scoped per key.** What is scoped is the action, the principal, and which records a caller can read.
- **Approval notifications are best effort.** One attempt, no retry. Refusal Records are the complete list.
- **Python verifies Ed25519 only.** ML-DSA-65 signatures are checked by the TypeScript verifier.
- **Inbound agents are not covered.** Parmana checks actions that leave through its gateway. It does not check an agent run by a customer or partner that calls one of your systems directly.

7.5 Reporting a vulnerability

Report vulnerabilities privately, as `SECURITY.md` in the repository describes. Do not open a public issue for a vulnerability.

8. Where to go next

- Run it locally: see "Run it locally" in the repository README.
- The full integration book, with step-by-step tutorials: docs.parmanasystems.com.
- Connect an agent: the SDK quickstarts for TypeScript and Python.
- Verify independently: the verification guide and the standalone offline verifier.